

Guilherme Molari

Engenheiro de Software Sênior • Web, mobile & jogos

Londrina, PR, Brasil yguilz123@gmail.com github.com/gmolari
linkedin.com/in/guilherme-molari-5203a8236

Engenheiro full stack sênior que constrói aplicações web escaláveis, APIs e apps mobile com Next.js, React, TypeScript, Node.js e .NET. Já entreguei plataformas em tempo real com alto tráfego, integrações de pagamento e de terceiros, backoffices, bibliotecas de componentes e melhorias de performance, sempre perto do design no Figma. Como Tech Lead na Ana Gaming liderei um time de cinco devs front-end e back-end, defini padrões de código e conduzi code review e mentoria. Também desenvolvo apps mobile (React Native, Kotlin, Java), jogos web com PixiJS e uso agentes de IA, Claude Code e MCP para automatizar fluxos de desenvolvimento.

EXPERIÊNCIA

GMaster • **Desenvolvedor Full Stack**

Evoluí um sistema interno já em produção de gestão de clientes e consultoria financeira. Criei endpoints REST em C# e .NET, desenvolvi telas em Next.js e participei da construção da arquitetura, além de modelagem de dados e consultas, integrações, melhorias de performance, correção de bugs e code review em toda a stack.

Ana Gaming • **Tech Lead**

Liderei um time de cinco devs front-end e back-end. Responsável por decisões de arquitetura, planejamento de entregas, padrões de código, code review e mentoria técnica.

StartBet • **Engenheiro Front-end**

Desenvolvi funcionalidades de gamificação e soluções para a plataforma de apostas: componentização, fluxos de aquisição e ferramentas que ajudaram o time a entregar mais rápido.

Multibet • **Engenheiro Front-end**

Desenvolvi funcionalidades do produto e de ferramentas internas: Multi-verso, backoffice de indicação, painéis de cadastro de popups e minigames e landing pages, partindo do Figma com React Query e Zustand.

Supernova Gaming • **Engenheiro Front-end Sênior**

Um dos pioneiros na plataforma Supernova Bet: recursos em tempo real, integrações REST e com provedores de pagamento, backoffices e uma biblioteca de componentes compartilhada. Depois assumi a responsabilidade pela camada de gamificação junto com a equipe e iniciei os projetos. O front do loop de gamificação foi 100% feito por mim.

PROJETOS

Supernova Bet

Supernova Gaming • Pioneiro na construção do front

Front da plataforma de apostas e cassino, construído desde o primeiro commit. Lobby de cassino, esportes, conta e pagamentos com atualizações em tempo real via Socket.IO, integrações REST com provedores de jogos e pagamento e uma biblioteca de componentes compartilhada.

Stack: Next.js, React, TypeScript, Socket.IO, Zustand, React Query

Gamification Loop

Supernova Gaming • Play4Tune • Front 100% por mim • iniciativa liderada com a equipe

A camada de retenção de uma plataforma de iGaming: missões, níveis, recompensas e minigames dentro do produto. O jogador avança por missões e níveis e ganha recompensas para gastar ou jogar. Os minigames e a UI vêm da biblioteca PGX UI, o estado de UI fica no Zustand e o de servidor no React Query, e tudo é configurado pelo backoffice de gamificação.

Stack: Next.js, React, TypeScript, PixiJS, Framer Motion, Zustand, React Query

Gamification Back Office

Supernova Gaming • Play4Tune • Construído inteiro por mim

O painel que controla todo o sistema de gamificação: a operação cria e ajusta cada missão, prêmio e jogo sem precisar de deploy. Missões e quests, loja, pontos, níveis, ranks VIP, torneios, bolões, minigames, jackpots, temas, onboarding e webhooks, tudo multi-moeda e multi-idioma, além do Jogador 360 e dashboards ao vivo. Os construtores são de arrastar e soltar, com prévia da tela real do app. Segurança em primeiro lugar: autenticação em duas etapas (TOTP com QR code e códigos de recuperação), sessão em cookie httpOnly assinado para o token nunca chegar ao navegador, token injetado no servidor e texto rico sanitizado. Um console de super admin provisiona novos tenants, e o painel se adapta à versão do sistema de cada um.

Stack: Next.js, React, TypeScript, React Query, TanStack Table, React Hook Form, Zod, Recharts, PixiJS, next-intl, PGX UI

PGX UI

Supernova Gaming • Play4Tune • Todo o código por mim • design do time de UX no Figma

A biblioteca de componentes da empresa: 81 componentes React, tema dinâmico e sete minigames, compartilhados pelo loop de gamificação, pelo backoffice e pela Supernova Bet. O tema de cada marca chega da API como tokens e desce por variáveis CSS até o Tailwind, então uma biblioteca veste todos os sites white-label. Escrevi todo o código, transformando os componentes do time de UX no Figma em React, ligados de volta ao Figma via Code Connect e documentados no Storybook. O motor de minigames roda em PixiJS 8 com o sorteio lacrado no servidor: roleta, caixa misteriosa e raspadinha em produção, além de mines, crash, pênalti e slot prontos no mesmo contrato, com som por instância e assets enviados pelo backoffice.

Stack: React, TypeScript, Tailwind, PixiJS, Framer Motion, Storybook, Figma, Vitest, npm

VortexVolley

Projeto pessoal • Solo, full stack

Um gerenciador de time de vôlei: app mobile, API e bot de WhatsApp. App em Expo com atualizações em tempo real via Socket.IO, API NestJS com JWT (access e refresh) e papéis, PostgreSQL com Prisma e um bot de WhatsApp que sincroniza a lista do jogo com o grupo.

Stack: Expo, React Native, NestJS, Prisma, PostgreSQL, Socket.IO

↗ Código-fonte

Bigette Fotos e Vídeos

Freelance • Solo, front e back end

Landing page de uma fotógrafa em Londrina, com back end administrativo próprio. Next.js 16 e React 19 com cenas 3D que só carregam em aparelhos capazes. Por trás, um painel admin autenticado (JWT e bcrypt) com PostgreSQL e Drizzle, upload de mídia no Vercel Blob e ordenação da galeria por arrastar e soltar. SEO completo.

Stack: Next.js, React, Tailwind, Three.js, Drizzle, PostgreSQL, Vercel Blob

↗ Código-fonte

Ad Monetization Engine

Laboratório de P&D • jogos mobile casuais • Solo: motor e três jogos

Um motor de monetização por anúncios para jogos mobile casuais. Para cada jogador e momento ele decide se mostra um anúncio recompensado, um intersticial ou um banner, para o estúdio faturar com anúncios sem queimar os jogadores. Ajustado como produto, não como código: limite de frequência, carência para jogadores novos, intersticial só no game over, revive recompensado, predefinições (recomendado, agressivo, sem anúncios), remote config e um registro ao vivo que explica cada decisão. AdMob com fallback automático para house ads de promoção cruzada, compra para remover anúncios, consentimento (UMP e LGPD) e proteção para menores já vêm prontos.

Stack: TypeScript, PixiJS v8, Vite, Capacitor, AdMob

Sweepstakes

Laboratório de P&D • Pesquisa e protótipo

Pesquisa de mercado e jurídica sobre cassinos sweepstakes e um protótipo original de cassino sweepstakes. Oito slots originais sobre um motor de rolos paramétrico em WebGL, moeda dupla, giros grátis, níveis VIP, passe de temporada, roda diária, missões, ferramentas de jogo responsável e um fluxo simulado de geobloqueio e KYC. Toda a arte é desenhada em código e roda na web e no Android.

Stack: JavaScript, PixiJS, Canvas, Android WebView

CityBuilder

Projeto pessoal • Solo

Um mod de city-builder e RTS para Minecraft 1.20. Feito em Kotlin sobre o Fabric, estendendo o mod MineFortress com mecânicas de construção de cidade e estratégia.

Stack: Kotlin, Fabric, Java

⚡ SKILLS

TypeScript • JavaScript • React • Next.js • React Native • Expo • PixiJS • Three.js • Framer Motion • Tailwind CSS • Zustand • React Query • Node.js • NestJS • Socket.IO • C# • ASP.NET • Entity Framework • Kotlin • Java • PostgreSQL • MySQL • MongoDB • Prisma • Drizzle • Capacitor • Figma • Claude Code • MCP